AutoCharge AI — Technical Summary (Team SOS, AgentVerse 2026 Robotics)

Problem and users. Drivers with limited mobility, fleet depots and unattended car parks cannot rely on someone being present to plug in an EV. Heavy CCS2 cables are awkward to handle, and inlets sit at different heights and angles on every car. AutoCharge AI is an embodied agent that finds the charging inlet with its own camera, plugs in, and stops safely when anything looks wrong.

What it does (MuJoCo 4.0 simulation, Google Robot from MuJoCo Menagerie, unmodified).

- **Perceive:** a camera on the connector (960×720) sees the car. OpenCV detects a fiducial beside the inlet, `solvePnP` estimates the inlet's 6-DoF pose, and the camera pose comes from the arm's own joint angles. A filter flags pose jumps (the car moved).
- **Decide:** a state machine (search → approach → align → insert → seated, plus retract, safe stop and failed) driven only by measurements: detection confidence and age, estimated alignment, plug speed, contact force, cable tension and strain-relief load.
- **Act:** a damped-least-squares Cartesian servo on the arm's original position actuators, with alignment-gated slow insertion (≤ 1 mm / 1°, then 5 mm/s). A simulated CCS2-style plug and inlet with real contact, and a flexible 2.9–4.5 m cable (1-D elastic rod) anchored at the charger and strain-relieved at the handle.
- **Adapt:** a separate scenario manager moves or obscures the car without telling the robot. The robot re-plans on its own observations.

Architecture. Camera render → ArUco + PnP (`perception.py`) → pose filter → agent and supervisor (`autonomy.py`) → insertion controller (`controller.py`) → MuJoCo actuators. Safety limits run every 2 ms control step: force 30 N pause / 70 N retract, cable tension 60 N, boot load 45 N, deflection 2.5 mm, emergency stop. An optional mobile base drives in from 3 m and docks (`mission.py`).

Where it runs / **hardware** / **latency.** Everything runs locally on a laptop CPU. There are no cloud calls and no GPU. Perception takes about 70–90 ms per frame at 10 Hz, control runs at 500 Hz, and the simulation runs at about 0.5–1× real time with perception on.

Measured results (`pytest tests`, simulation only):

- Pose estimate 0.2–0.6 mm and 0.1–0.2° from ground truth at 0.3 m.
- Seated from camera only, with 0.1–0.3 mm real error.
- Random parking (±3 cm, ±4 cm sideways, ±3 cm height, ±4°): seated.
- Car rolls 3 cm and turns 3° mid-approach: detected within 0.2–0.3 s, re-planned, seated.
- Inlet view blocked: the arm never moves.
- Car moved while engaged: safe stop.
- 8 mm calibration error injected: the robot follows its (wrong) perception and does not seat, which shows actions come from perception, not hidden data.
- **Held-out benchmark (10 unseen random parkings, half with the car moving mid-approach):** the autonomous agent docked **10/10**. The same task with a fixed-coordinate (pre-programmed) controller docked **0/10**. It jammed on the rim, and the force limits made it back out safely.
- Simulator ground truth corrupted by 2 cm: still docks, so the robot never reads hidden coordinates.

Limitations.

- The inlet is found through a fiducial marker, not by recognising the bare inlet. The teammate's learned detector can replace the detector behind the same interface.
- The last few millimetres of insertion run on the last estimate plus force sensing.
- Simplified CCS2 geometry: no latch or electrical model, and insertion force is far lower than real.
- The drive-in base uses true odometry, with no wheel dynamics.
- After a large car move, the first insertion attempts can drift and be retried.
- Dimmed lighting is tested (docks at 15%, refuses at 3%); glare and lens effects are not.

Most critical failure mode and safety. The most dangerous case is pushing a plug into the wrong place, or dragging the car or the cable. The robot never moves without a confident, fresh detection. It pauses when perception goes stale, re-plans if the car moves before engagement, and holds still (safe stop, no further force) if the car moves after engagement. It backs out on force, deflection or cable limits, and retries are bounded. These thresholds are tuned for the simulation and are not certified real-world values.